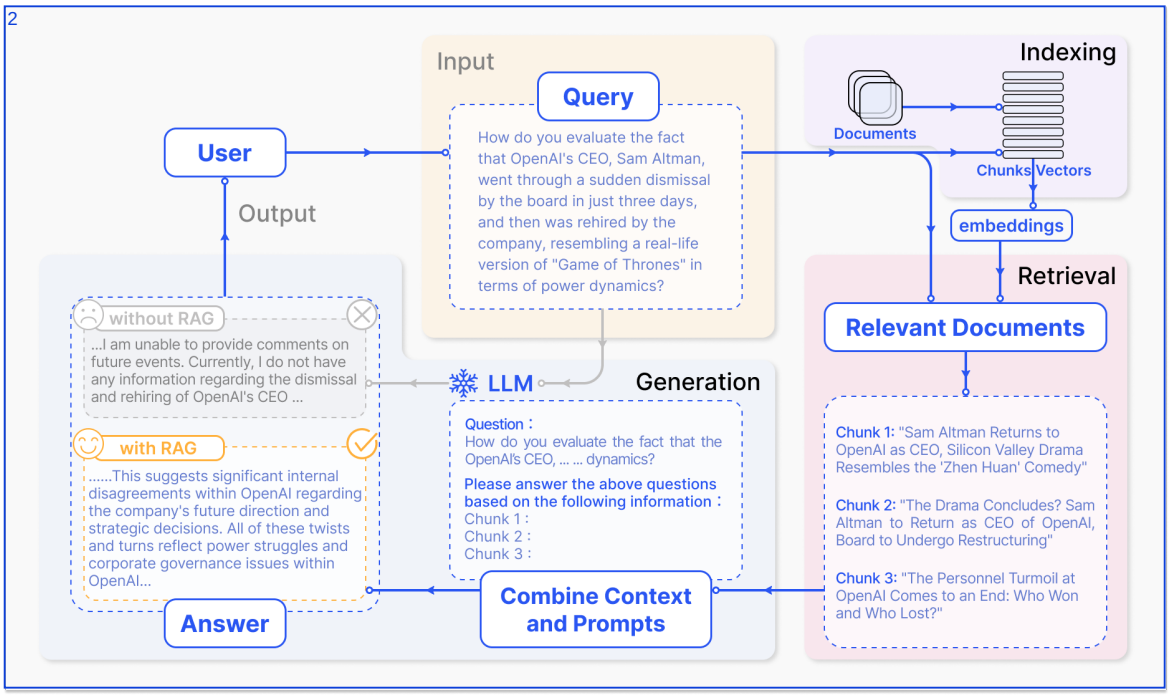


Fig. 2. A representative instance of the RAG process applied to question answering. It mainly consists of 3 steps. 1) Indexing. Documents are split into chunks, encoded into vectors, and stored in a vector database. 2) Retrieval. Retrieve the Top k chunks most relevant to the question based on semantic similarity. 3) Generation. Input the original question and the retrieved chunks together into LLM to generate the final answer.

widespread adoption of ChatGPT. The Naive RAG follows a traditional process that includes indexing, retrieval, and generation, which is also characterized as a "Retrieve-Read" framework [7].

Indexing starts with the cleaning and extraction of raw data in diverse formats like PDF, HTML, Word, and Markdown, which is then converted into a uniform plain text format. To accommodate the context limitations of language models, text is segmented into smaller, digestible chunks. Chunks are then encoded into vector representations using an embedding model and stored in vector database. This step is crucial for enabling efficient similarity searches in the subsequent retrieval phase.

Retrieval. Upon receipt of a user query, the RAG system employs the same encoding model utilized during the indexing phase to transform the query into a vector representation. It then computes the similarity scores between the query vector and the vector of chunks within the indexed corpus. The system prioritizes and retrieves the top K chunks that demonstrate the greatest similarity to the query. These chunks are subsequently used as the expanded context in prompt.

Generation. The posed query and selected documents are synthesized into a coherent prompt to which a large language model is tasked with formulating a response. The model's approach to answering may vary depending on task-specific criteria, allowing it to either draw upon its inherent parametric knowledge or restrict its responses to the information contained within the provided documents. In cases of ongoing dialogues, any existing conversational history can be integrated into the prompt, enabling the model to engage in multi-turn dialogue interactions effectively.

However, Naive RAG encounters notable drawbacks:

Retrieval Challenges. The retrieval phase often struggles with precision and recall, leading to the selection of misaligned or irrelevant chunks, and the missing of crucial information.

Generation Difficulties. In generating responses, the model may face the issue of hallucination, where it produces content not supported by the retrieved context. This phase can also suffer from irrelevance, toxicity, or bias in the outputs, detracting from the quality and reliability of the responses.

Augmentation Hurdles. Integrating retrieved information with the different task can be challenging, sometimes resulting in disjointed or incoherent outputs. The process may also encounter redundancy when similar information is retrieved from multiple sources, leading to repetitive responses. Determining the significance and relevance of various passages and ensuring stylistic and tonal consistency add further complexity. Facing complex issues, a single retrieval based on the original query may not suffice to acquire adequate context information.

Moreover, there's a concern that generation models might overly rely on augmented information, leading to outputs that simply echo retrieved content without adding insightful or synthesized information.

B. Advanced RAG

Advanced RAG introduces specific improvements to overcome the limitations of Naive RAG. Focusing on enhancing retrieval quality, it employs pre-retrieval and post-retrieval strategies. To tackle the indexing issues, Advanced RAG refines its indexing techniques through the use of a sliding window approach, fine-grained segmentation, and the incorporation of metadata. Additionally, it incorporates several optimization methods to streamline the retrieval process [8].

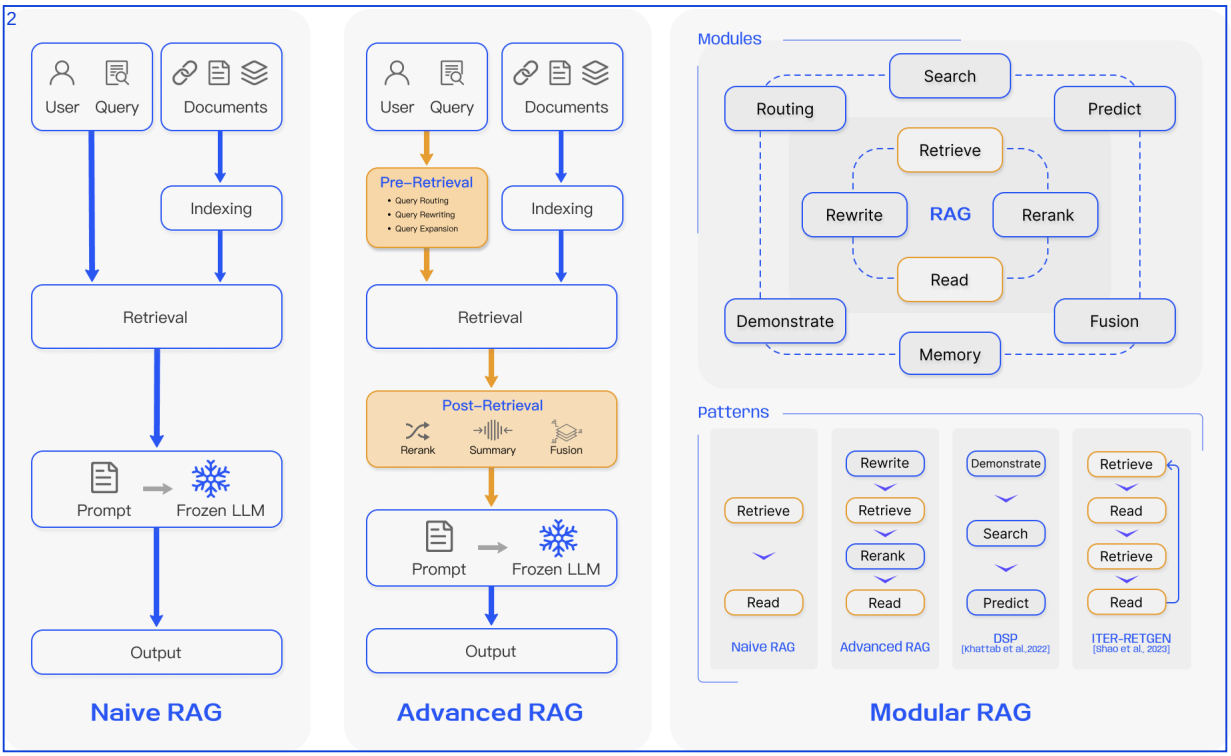


Fig. 3. Comparison between the three paradigms of RAG. (Left) Naive RAG mainly consists of three parts: indexing, retrieval and generation. (Middle) Advanced RAG proposes multiple optimization strategies around pre-retrieval and post-retrieval, with a process similar to the Naive RAG, still following a chain-like structure. (Right) Modular RAG inherits and develops from the previous paradigm, showcasing greater flexibility overall. This is evident in the introduction of multiple specific functional modules and the replacement of existing modules. The overall process is not limited to sequential retrieval and generation; it includes methods such as iterative and adaptive retrieval.

Pre-retrieval process. In this stage, the primary focus is on optimizing the indexing structure and the original query. The goal of optimizing indexing is to enhance the quality of the content being indexed. This involves strategies: enhancing data granularity, optimizing index structures, adding metadata, alignment optimization, and mixed retrieval. While the goal of query optimization is to make the user's original question clearer and more suitable for the retrieval task. Common methods include query rewriting query transformation, query expansion and other techniques [7], [9]–[11].

Post-Retrieval Process. Once relevant context is retrieved, it's crucial to integrate it effectively with the query. The main methods in post-retrieval process include rerank chunks and context compressing. Re-ranking the retrieved information to relocate the most relevant content to the edges of the prompt is a key strategy. This concept has been implemented in frameworks such as LlamaIndex[2], LangChain[3], and HayStack [12]. Feeding all relevant documents directly into LLMs can lead to information overload, diluting the focus on key details with irrelevant content.To mitigate this, post-retrieval efforts concentrate on selecting the essential information, emphasizing critical sections, and shortening the context to be processed.

[2]https://www.llamaindex.ai
[3]https://www.langchain.com/

C. Modular RAG

The modular RAG architecture advances beyond the former two RAG paradigms, offering enhanced adaptability and versatility. It incorporates diverse strategies for improving its components, such as adding a search module for similarity searches and refining the retriever through fine-tuning. Innovations like restructured RAG modules [13] and rearranged RAG pipelines [14] have been introduced to tackle specific challenges. The shift towards a modular RAG approach is becoming prevalent, supporting both sequential processing and integrated end-to-end training across its components. Despite its distinctiveness, Modular RAG builds upon the foundational principles of Advanced and Naive RAG, illustrating a progression and refinement within the RAG family.

1) New Modules: The Modular RAG framework introduces additional specialized components to enhance retrieval and processing capabilities. The Search module adapts to specific scenarios, enabling direct searches across various data sources like search engines, databases, and knowledge graphs, using LLM-generated code and query languages [15]. RAG-Fusion addresses traditional search limitations by employing a multi-query strategy that expands user queries into diverse perspectives, utilizing parallel vector searches and intelligent re-ranking to uncover both explicit and transformative knowledge [16]. The Memory module leverages the LLM's memory to guide retrieval, creating an unbounded memory pool that

TABLE I
SUMMARY OF RAG METHODS

Method	Retrieval Source	Retrieval Data Type	Retrieval Granularity	Augmentation Stage	Retrieval process
CoG [29]	Wikipedia	Text	Phrase	Pre-training	Iterative
DenseX [30]	FactoidWiki	Text	Proposition	Inference	Once
EAR [31]	Dataset-base	Text	Sentence	Tuning	Once
UPRISE [20]	Dataset-base	Text	Sentence	Tuning	Once
RAST [32]	Dataset-base	Text	Sentence	Tuning	Once
Self-Mem [17]	Dataset-base	Text	Sentence	Tuning	Iterative
FLARE [24]	Search Engine,Wikipedia	Text	Sentence	Tuning	Adaptive
PGRA [33]	Wikipedia	Text	Sentence	Inference	Once
FILCO [34]	Wikipedia	Text	Sentence	Inference	Once
RADA [35]	Dataset-base	Text	Sentence	Inference	Once
Filter-rerank [36]	Synthesized dataset	Text	Sentence	Inference	Once
R-GQA [37]	Dataset-base	Text	Sentence Pair	Tuning	Once
LLM-R [38]	Dataset-base	Text	Sentence Pair	Inference	Iterative
TIGER [39]	Dataset-base	Text	Item-base	Pre-training	Once
LM-Indexer [40]	Dataset-base	Text	Item-base	Tuning	Once
BEQUE [9]	Dataset-base	Text	Item-base	Tuning	Once
CT-RAG [41]	Synthesized dataset	Text	Item-base	Tuning	Once
Atlas [42]	Wikipedia, Common Crawl	Text	Chunk	Pre-training	Iterative
RAVEN [43]	Wikipedia	Text	Chunk	Pre-training	Once
RETRO++ [44]	Pre-training Corpus	Text	Chunk	Pre-training	Iterative
INSTRUCTRETRO [45]	Pre-training corpus	Text	Chunk	Pre-training	Iterative
RRR [7]	Search Engine	Text	Chunk	Tuning	Once
RA-e2e [46]	Dataset-base	Text	Chunk	Tuning	Once
PROMPTAGATOR [21]	BEIR	Text	Chunk	Tuning	Once
AAR [47]	MSMARCO,Wikipedia	Text	Chunk	Tuning	Once
RA-DIT [27]	Common Crawl,Wikipedia	Text	Chunk	Tuning	Once
RAG-Robust [48]	Wikipedia	Text	Chunk	Tuning	Once
RA-Long-Form [49]	Dataset-base	Text	Chunk	Tuning	Once
CoN [50]	Wikipedia	Text	Chunk	Tuning	Once
Self-RAG [25]	Wikipedia	Text	Chunk	Tuning	Adaptive
BGM [26]	Wikipedia	Text	Chunk	Inference	Once
CoQ [51]	Wikipedia	Text	Chunk	Inference	Iterative
Token-Elimination [52]	Wikipedia	Text	Chunk	Inference	Once
PaperQA [53]	Arxiv,Online Database,PubMed	Text	Chunk	Inference	Iterative
NoiseRAG [54]	FactoidWiki	Text	Chunk	Inference	Once
IAG [55]	Search Engine,Wikipedia	Text	Chunk	Inference	Once
NoMIRACL [56]	Wikipedia	Text	Chunk	Inference	Once
ToC [57]	Search Engine,Wikipedia	Text	Chunk	Inference	Recursive
SKR [58]	Dataset-base,Wikipedia	Text	Chunk	Inference	Adaptive
ITRG [59]	Wikipedia	Text	Chunk	Inference	Iterative
RAG-LongContext [60]	Dataset-base	Text	Chunk	Inference	Once
ITER-RETGEN [14]	Wikipedia	Text	Chunk	Inference	Iterative
IRCoT [61]	Wikipedia	Text	Chunk	Inference	Recursive
LLM-Knowledge-Boundary [62]	Wikipedia	Text	Chunk	Inference	Once
RAPTOR [63]	Dataset-base	Text	Chunk	Inference	Recursive
RECITE [22]	LLMs	Text	Chunk	Inference	Once
ICRALM [64]	Pile,Wikipedia	Text	Chunk	Inference	Iterative
Retrieve-and-Sample [65]	Dataset-base	Text	Doc	Tuning	Once
Zemi [66]	C4	Text	Doc	Tuning	Once
CRAG [67]	Arxiv	Text	Doc	Inference	Once
1-PAGER [68]	Wikipedia	Text	Doc	Inference	Iterative
PRCA [69]	Dataset-base	Text	Doc	Inference	Once
QLM-Doc-ranking [70]	Dataset-base	Text	Doc	Inference	Once
Recomp [71]	Wikipedia	Text	Doc	Inference	Once
DSP [23]	Wikipedia	Text	Doc	Inference	Iterative
RePLUG [72]	Pile	Text	Doc	Inference	Once
ARM-RAG [73]	Dataset-base	Text	Doc	Inference	Iterative
GenRead [13]	LLMs	Text	Doc	Inference	Iterative
UniMS-RAG [74]	Dataset-base	Text	Multi	Tuning	Once
CREA-ICL [19]	Dataset-base	Crosslingual,Text	Sentence	Inference	Once
PKG [75]	LLM	Tabular,Text	Chunk	Inference	Once
SANTA [76]	Dataset-base	Code,Text	Item	Pre-training	Once
SURGE [77]	Freebase	KG	Sub-Graph	Tuning	Once
MK-ToD [78]	Dataset-base	KG	Entity	Tuning	Once
Dual-Feedback-ToD [79]	Dataset-base	KG	Entity Sequence	Tuning	Once
KnowledGPT [15]	Dataset-base	KG	Triplet	Inference	Muti-time
FABULA [80]	Dataset-base,Graph	KG	Entity	Inference	Once
HyKGE [81]	CMeKG	KG	Entity	Inference	Once
KALMV [82]	Wikipedia	KG	Triplet	Inference	Iterative
RoG [83]	Freebase	KG	Triplet	Inference	Iterative
G-Retriever [84]	Dataset-base	TextGraph	Sub-Graph	Inference	Once